
LAPORAN PRAKTIKUM TEKNOLOGI IOT II



Disusun Oleh:

Nama: M Riki Pratama

Kelas: Teknologi Rekayasa Internet (3C)

NPM: 25782072

Dosen:

Agus Ambarwari, S.Pd., M.Kom.

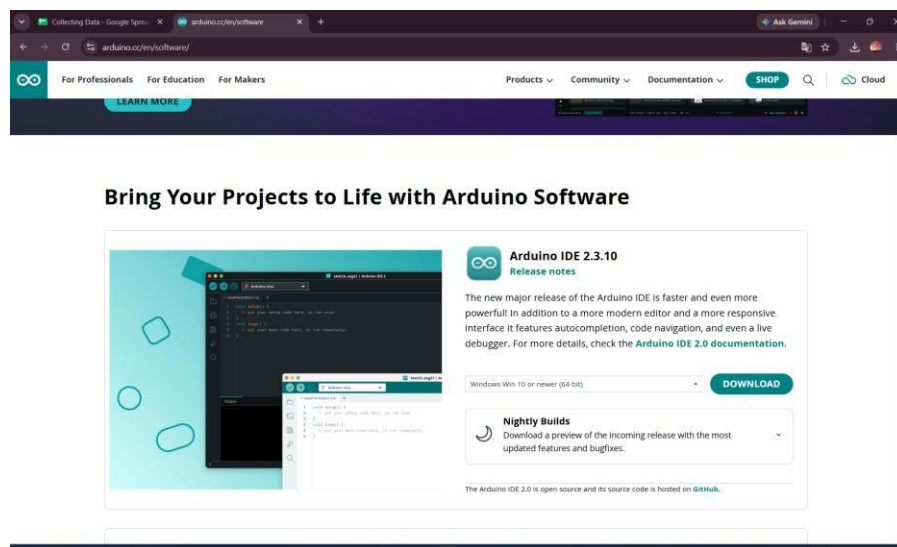
TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG 2026

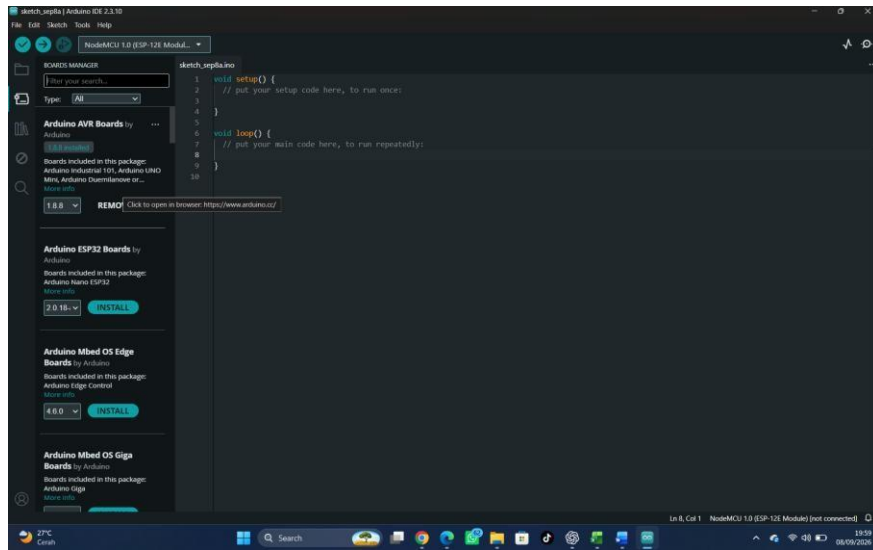
1.4.2 Konfigurasi dan Instalasi Lingkungan Pengembangan (Arduino IDE)

Instalasi Arduino IDE dilakukan sebagai tahap awal sebelum melakukan pemrograman pada mikrokontroler. Pertama, Arduino IDE dapat diunduh melalui situs resmi Arduino sesuai dengan sistem operasi yang digunakan, kemudian file installer yang telah selesai diunduh dijalankan dengan melakukan *double click*. Setelah itu, pengguna mengikuti proses instalasi dengan menyetujui lisensi, memilih komponen dan lokasi instalasi, lalu menekan tombol Install hingga proses selesai. Setelah Arduino IDE berhasil terpasang, aplikasi dapat dibuka melalui Start Menu atau shortcut pada desktop.



Sebelum memprogram, instal *board package* ESP8266 pada perangkat lunak Arduino IDE agar mikrokontroler dikenali.

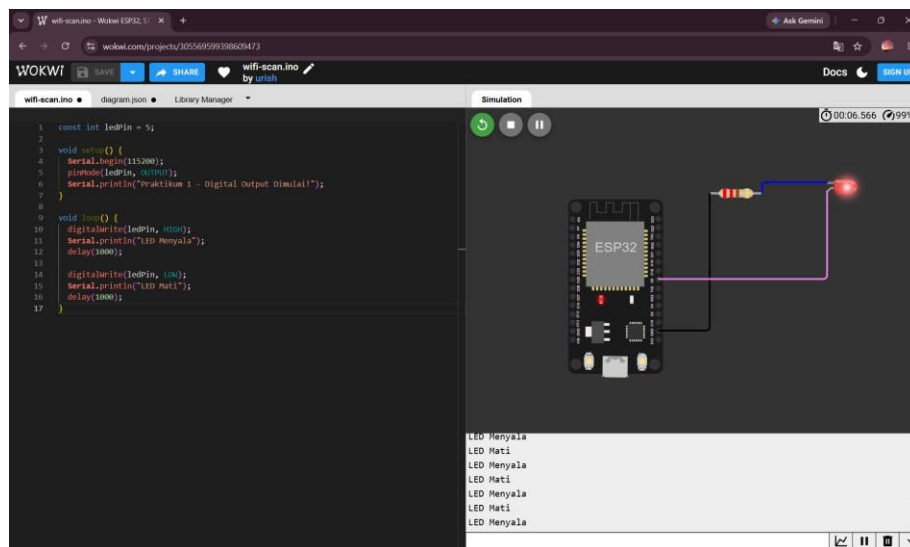
1. Buka aplikasi Arduino IDE.
2. Buka menu File > Preferences.
3. Pada kolom Additional Boards Manager URLs, tempelkan tautan berikut: http://arduino.esp8266.com/stable/package_esp8266com_index.json
4. Buka menu Tools > Board > Boards Manager...
5. Ketik "esp8266" di kolom pencarian, kemudian klik Install pada paket esp8266 by ESP8266 Community. Tunggu hingga proses unduhan selesai.
6. Konfigurasi parameter *board* Anda pada menu Tools:
 - Board: Pilih NodeMCU 1.0 (ESP-12E Module).
 - Port: Pilih *port COM* yang muncul saat NodeMCU dihubungkan ke laptop via USB.



1.5 Praktikum

1.5.1 Praktikum 1: *Digital Output* (Menyalakan LED)

Praktikum ini bertujuan memprogram NodeMCU agar mengalirkan tegangan 3.3V (HIGH) secara terprogram untuk menyalakan komponen eksternal.



Gambar Praktikum 1

A. Langkah Kerja Rangkaian

1. Membuka Wokwi melalui browser, kemudian membuat project baru dengan menggunakan board ESP32 DevKit.
2. Menambahkan komponen LED dan resistor ke dalam area rangkaian.

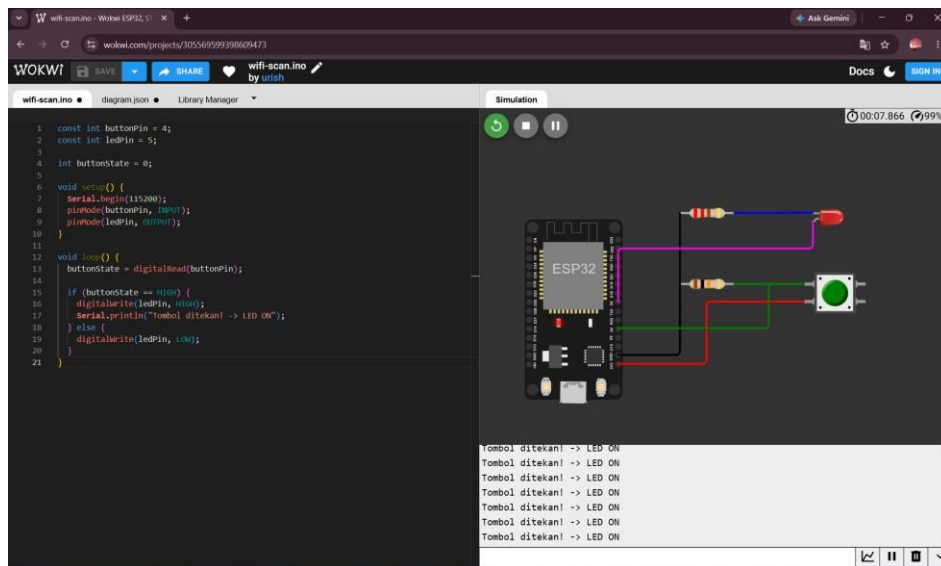
3. Menghubungkan salah satu kaki resistor ke pin GPIO 5 atau D5 pada ESP32.
4. Menghubungkan kaki resistor yang lainnya ke kaki anoda LED.
5. Menghubungkan kaki katoda LED ke pin GND pada ESP32.
6. Memeriksa kembali seluruh kabel dan posisi komponen untuk memastikan tidak ada sambungan yang terlepas atau salah.
7. Setelah rangkaian selesai, menjalankan simulasi untuk melihat apakah LED dapat menyala dan mati sesuai dengan program.

B. Langkah Kerja Pemrograman

1. Membuka bagian editor kode pada project Wokwi yang telah dibuat.
2. Menentukan pin yang digunakan untuk LED dengan menuliskan `const int ledPin = 5;`. Pada program ini LED menggunakan pin GPIO 5.
3. Membuat fungsi `setup()` sebagai bagian awal program yang hanya dijalankan satu kali.
4. Mengaktifkan komunikasi Serial dengan kecepatan 115200 baud menggunakan perintah `Serial.begin(115200);`.
5. Mengatur GPIO 5 sebagai output menggunakan perintah `pinMode(ledPin, OUTPUT);`.
6. Menampilkan tulisan "Praktikum 1 - Digital Output Dimulai!" pada Serial Monitor sebagai tanda bahwa program sudah mulai dijalankan.
7. Membuat fungsi `loop()` untuk menjalankan perintah secara berulang.
8. Memberikan kondisi HIGH pada pin LED menggunakan `digitalWrite(ledPin, HIGH);` sehingga LED menyala.
9. Menampilkan tulisan "LED Menyala" pada Serial Monitor dan memberikan waktu tunggu selama satu detik.
10. Setelah satu detik, memberikan kondisi LOW pada pin LED menggunakan `digitalWrite(ledPin, LOW);` sehingga LED mati.
11. Menampilkan tulisan "LED Mati" pada Serial Monitor dan memberikan waktu tunggu selama satu detik.
12. Menjalankan simulasi dan mengamati perubahan kondisi LED serta tulisan yang muncul pada Serial Monitor.
13. Jika rangkaian dan program sudah benar, LED akan menyala selama satu detik kemudian mati selama satu detik. Proses tersebut akan terus berulang selama simulasi berjalan.

1.5.2 Praktikum 2: *Digital Input* (Membaca Push Button)

Praktikum ini bertujuan mendeteksi interaksi fisik dengan membaca status tegangan dari sebuah tombol, memanfaatkan rangkaian resistor *pull-down* eksternal



Gambar Praktikum 2

A. Langkah Kerja Rangkaian

1. Membuka Wokwi melalui browser dan membuat project baru menggunakan board ESP32 DevKit.
2. Menambahkan komponen LED, push button, dan dua buah resistor ke dalam area kerja.
3. Menghubungkan pin GPIO 5 pada ESP32 ke resistor LED.
4. Menghubungkan resistor tersebut ke kaki anoda LED, kemudian kaki katoda LED dihubungkan ke GND.
5. Menghubungkan salah satu kaki push button ke sumber tegangan 3.3V pada ESP32.
6. Menghubungkan kaki push button lainnya ke GPIO 4. Pin ini digunakan untuk membaca kondisi tombol.
7. Memasang resistor pada jalur GPIO 4 sebagai pull-down, kemudian menghubungkannya ke GND. Resistor ini berfungsi agar kondisi pin tetap LOW ketika tombol tidak ditekan.
8. Memeriksa kembali seluruh sambungan rangkaian, terutama pin GPIO 4, GPIO 5, 3.3V, dan GND.

9. Menjalankan simulasi Wokwi dan mencoba menekan push button untuk melihat perubahan kondisi LED.

B. Langkah Kerja Pemrograman

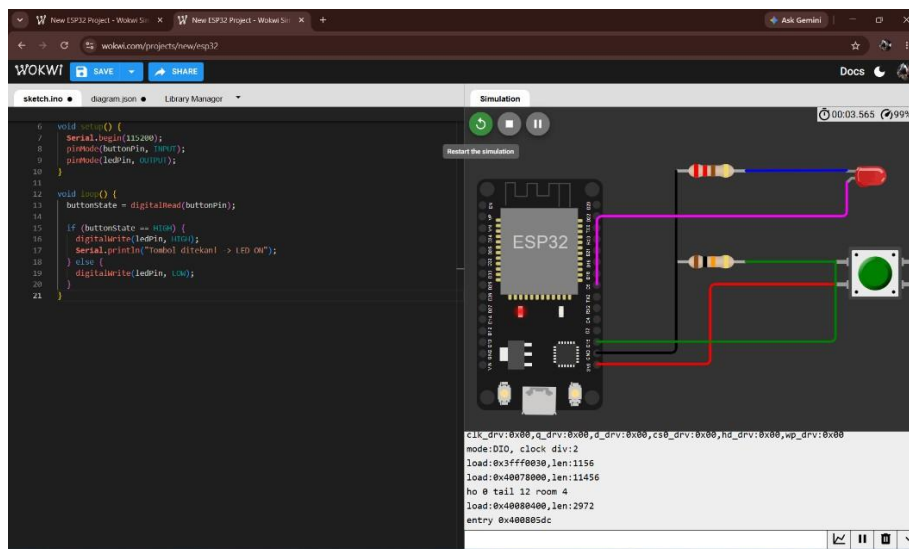
1. Membuka editor kode pada project Wokwi.
2. Menentukan pin yang digunakan untuk push button dan LED dengan program:
3. `const int buttonPin = 4;`
`const int ledPin = 5;`
4. Membuat variabel `buttonState` untuk menyimpan kondisi tombol:
`int buttonState = 0;`
5. Membuat fungsi `setup()` untuk melakukan pengaturan awal program.
6. Mengaktifkan komunikasi Serial menggunakan `Serial.begin(115200);` agar kondisi tombol dapat dilihat melalui Serial Monitor.
7. Mengatur GPIO 4 sebagai INPUT karena pin tersebut digunakan untuk membaca kondisi push button.
8. Mengatur GPIO 5 sebagai OUTPUT karena pin tersebut digunakan untuk mengendalikan LED.
9. Membuat fungsi `loop()` agar ESP32 dapat membaca kondisi push button secara terus-menerus.
10. Membaca kondisi tombol menggunakan perintah:
`buttonState = digitalRead(buttonPin);`
11. Membuat kondisi if untuk memeriksa apakah tombol sedang ditekan. Jika nilai `buttonState` bernilai HIGH, ESP32 akan menyalakan LED menggunakan:
`digitalWrite(ledPin, HIGH);`
12. Ketika tombol ditekan, program juga menampilkan tulisan "Tombol ditekan! -> LED ON" pada Serial Monitor.
13. Jika tombol tidak ditekan atau nilai `buttonState` bernilai LOW, LED dimatikan menggunakan:
`digitalWrite(ledPin, LOW);`
14. Menjalankan simulasi dan menekan push button. Ketika tombol ditekan, LED akan menyala, sedangkan ketika tombol dilepas, LED akan mati.
15. Mengamati Serial Monitor untuk memastikan pesan kondisi tombol muncul sesuai dengan keadaan push button.

1.6 Latihan

Kerjakan latihan berikut untuk menguji pemahaman Anda. Jelaskan hasilnya di dalam laporan!

- Latihan 1 (Pemahaman Logika Pin): Berdasarkan tabel Referensi Pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / *pull-down*), namun menyambungkan tombol tersebut ke pin D8 (GPIO 15), lalu menekan tombol tersebut *tepat* pada detik di mana NodeMCU baru diberi daya?

Jika tombol pada rangkaian Praktikum 2 dipasang pada D8/GPIO15 dan tombol ditekan tepat saat NodeMCU dinyalakan, hal tersebut dapat mengganggu proses booting. D8 atau GPIO15 merupakan salah satu pin yang digunakan saat proses awal ESP8266. Jadi, jika kondisi pin tidak sesuai ketika perangkat mulai menyala, NodeMCU bisa gagal melakukan booting dengan normal atau program tidak berjalan seperti biasanya.

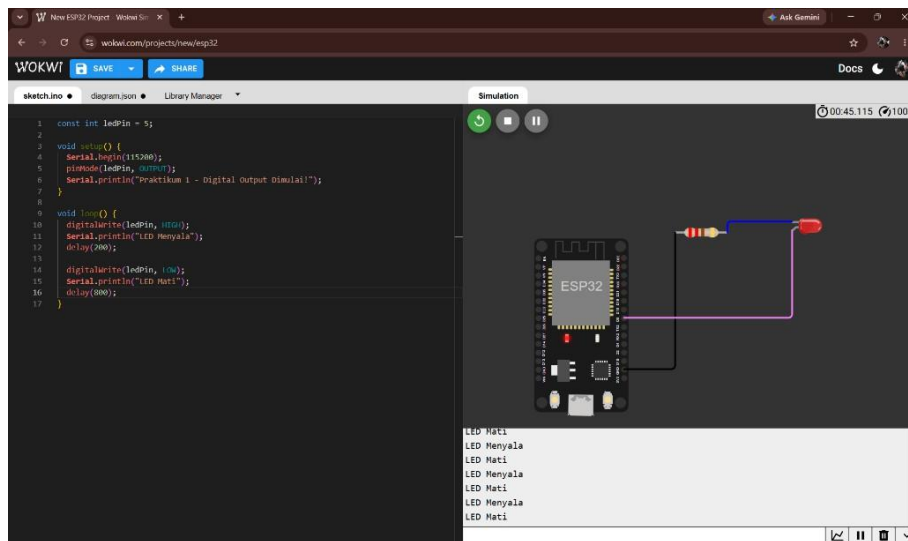


Gambar Latihan 1

- Latihan 2 (Implementasi Kode): Modifikasi kode pada Praktikum 1 agar LED berkedip secara *tidak simetris*, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi delay() yang harus Anda ubah.

Serial.println("Led Menyala/Led Mati");

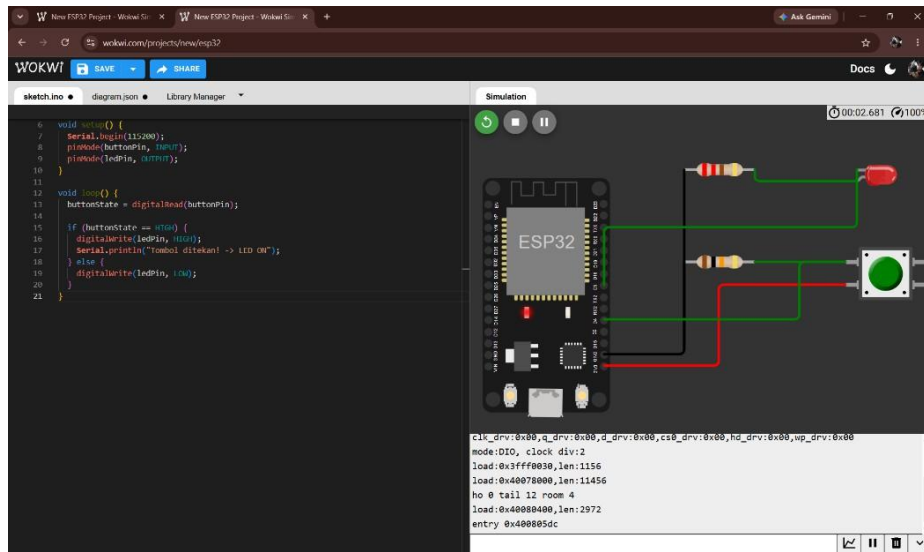
Delay(200);



Gambar Latihan 2

- Latihan 3 (Analisis Sirkuit): Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami *error* kompilasi? Mengapa LED tidak menyala?

Tidak akan terjadi compile error karena posisi LED merupakan bagian dari rangkaian hardware, bukan kesalahan pada program. Namun, LED tidak akan menyala karena LED memiliki polaritas. Saat dipasang terbalik, arus tidak dapat mengalir melalui LED dengan arah yang benar.



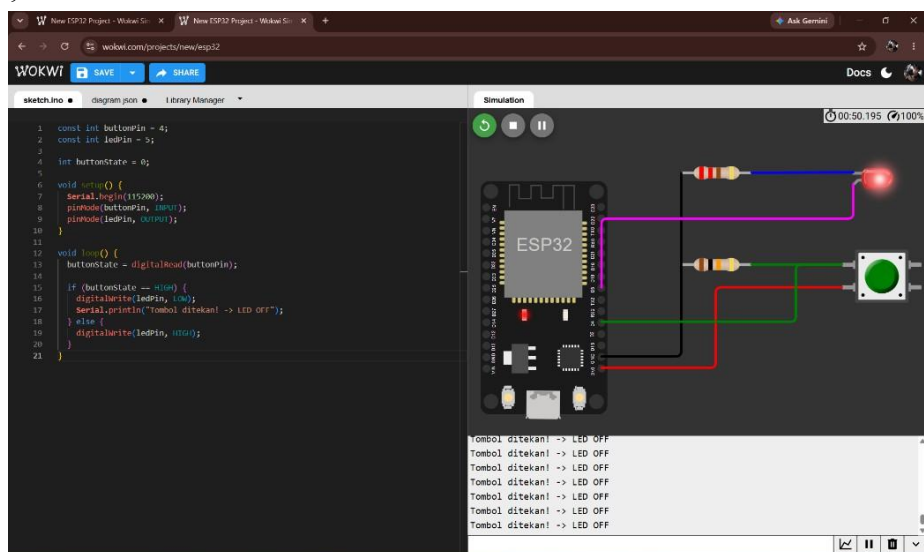
Gambar Latihan 3

- Latihan 4 (Pengembangan Algoritma): Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED menyala. Saat tombol DITEKAN, LED justru mati."

```

if (buttonState == HIGH) {
  digitalWrite(ledPin, LOW);
} else {
  digitalWrite(ledPin, HIGH);
}

```



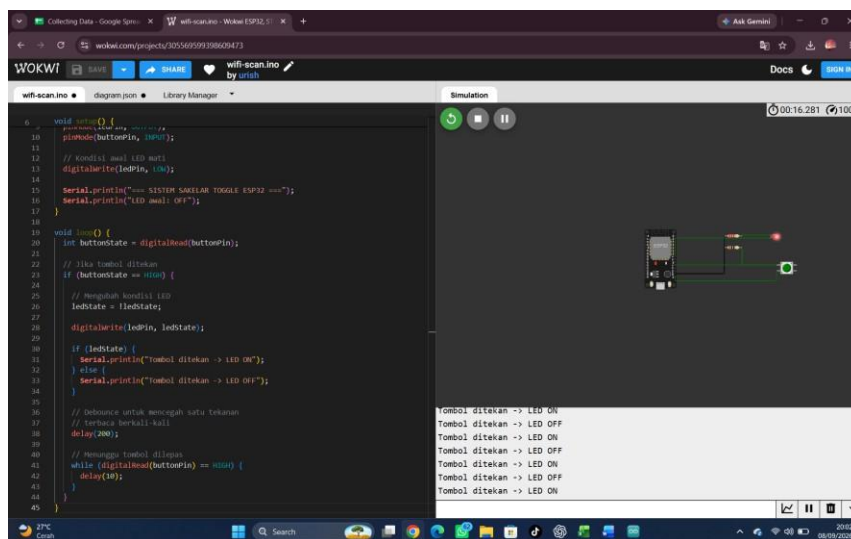
Gambar Latihan 4

1.7 Tugas

Tugas ini harus dikerjakan secara mandiri untuk mengukur kemampuan integrasi dan logika pemrograman Anda.

Modifikasi Sistem Sakelar *Toggle (Latching)*

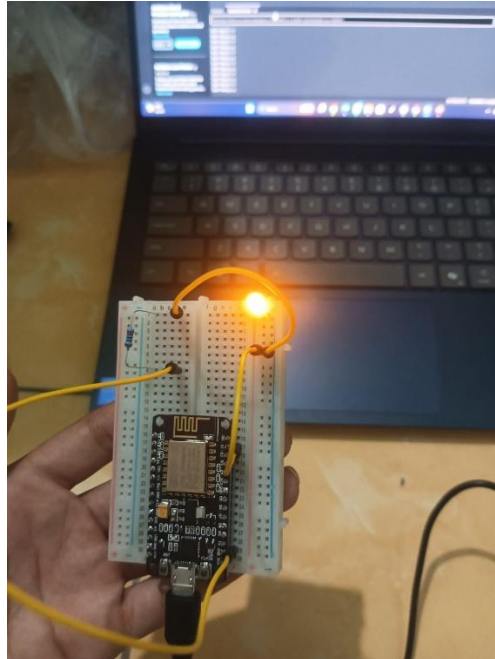
- Tujuan: Mampu mengimplementasikan logika pelacakan status (*state tracking*) dan membedakan antara perilaku tombol *momentary* dengan *latching*.
- Tingkat Kesulitan: Menengah
- Instruksi:
 - i. Gunakan konfigurasi *hardware* yang persis sama dengan Praktikum 2 (Rangkaian *Pull-Down*).
 - ii. Modifikasi kode program sehingga tombol berfungsi layaknya sakelar lampu ruangan:
 - Saat sistem pertama dinyalakan, LED dalam keadaan mati.
 - Jika tombol ditekan 1 kali *lalu dilepas*, LED menyala dan tetap menyala.
 - Jika tombol ditekan 1 kali lagi *lalu dilepas*, LED mati dan tetap mati.
 - iii. *Petunjuk*: Gunakan variabel penyimpanan status (misalnya tipe boolean atau int) untuk melacak apakah lampu saat ini sedang ON atau OFF. Anda juga membutuhkan instruksi `delay()` singkat (sekitar 200 milidetik) tepat setelah mendeteksi tombol ditekan, untuk mengatasi fenomena pantulan mekanis (*debouncing* pada tombol fisik).



Gambar Tugas

1.5 Praktikum

1.5.1 Praktikum 1: *Digital Output* (Menyalakan LED)

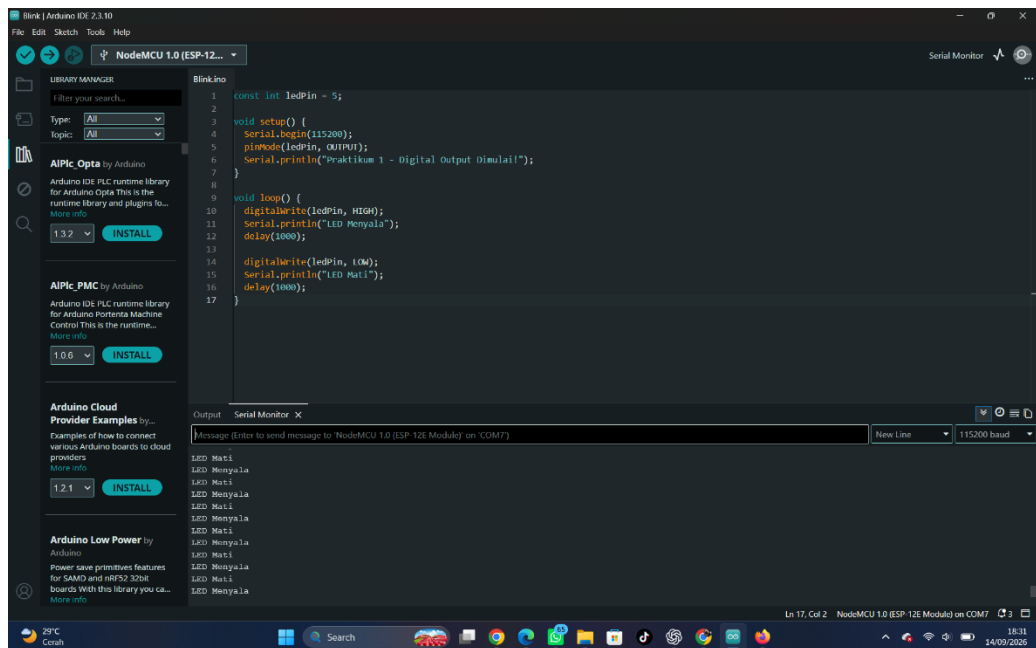


Langkah Kerja Rangkaian:

1. Pasang NodeMCU pada *breadboard*.
2. Pasang LED pada baris kosong di *breadboard*. Perhatikan kakinya:
 - Kaki yang lebih panjang adalah Anoda (Positif).
 - Kaki yang lebih pendek (dan terdapat sisi datar di tepi plastik kepala LED) adalah Katoda (Negatif).
3. Hubungkan pin D1 (GPIO 5) dari NodeMCU ke kaki Anoda LED menggunakan kabel *jumper*.
4. Hubungkan kaki Katoda LED ke salah satu ujung Resistor 220 Ohm.
5. Hubungkan ujung Resistor lainnya ke jalur GND pada NodeMCU.
(Catatan: Resistor berfungsi sebagai pembatas arus agar LED tidak terbakar).

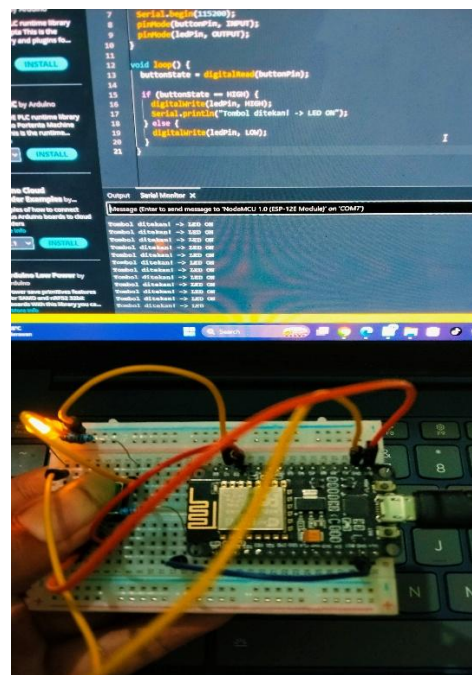
Langkah Kerja Pemrograman:

1. Buka Arduino IDE, klik File > New Sketch.
2. Tuliskan kode program



3. Klik tombol **Upload** (ikon panah ke kanan). Tunggu hingga muncul notifikasi *Done uploading*.
4. Buka **Serial Monitor** (ikon kaca pembesar di sudut kanan atas IDE).
5. Ubah pengaturan *baud rate* pada pojok kanan bawah Serial Monitor menjadi **115200 baud**.
6. Amati hasilnya: LED fisik berkedip setiap 1 detik selaras dengan teks status yang dicetak pada Serial Monitor.

1.5.2 Praktikum 2: *Digital Input (Membaca Push Button)*



Langkah Kerja Rangkaian:

1. Biarkan rangkaian LED pada pin **D1** dari Praktikum 1 tetap terpasang.
2. Pasang sebuah *push button* pada *breadboard*, melintasi garis pemisah/parit tengah.
3. Hubungkan **kaki pertama** *push button* ke pin **3V3** (3.3V) pada NodeMCU.
4. Hubungkan **kaki kedua** *push button* ke pin **D2 (GPIO 4)** NodeMCU.
5. Pada **kaki kedua** yang sama, pasang sebuah **Resistor 10k Ohm**, lalu hubungkan ujung lain dari resistor tersebut ke jalur **GND** NodeMCU.

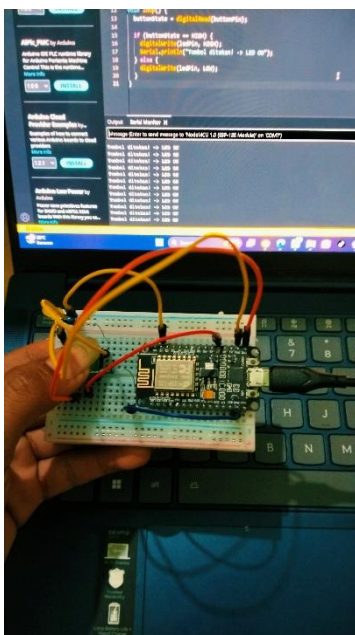
Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*) pada Arduino IDE.
2. Tuliskan kode program
3. **Upload** program ke dalam mikrokontroler.
4. Buka **Serial Monitor**.
5. Uji fungsionalitasnya: Saat tombol ditekan dan ditahan, LED akan menyala terang. Saat tombol dilepaskan, LED langsung padam.

1.6 Latihan

Kerjakan latihan berikut untuk menguji pemahaman Anda. Jelaskan hasilnya di dalam laporan!

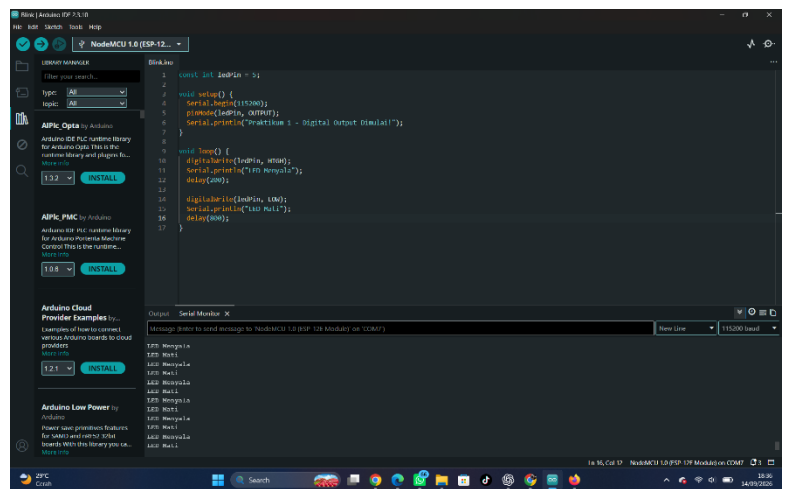
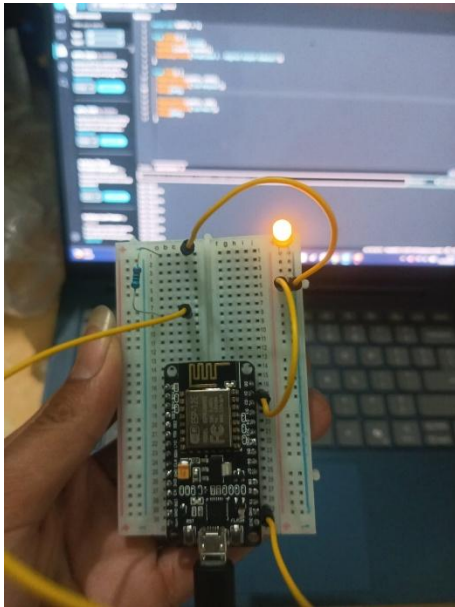
Latihan 1 (Pemahaman Logika Pin): Berdasarkan tabel Referensi Pinout di subbab 1.3.4, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / *pull-down*), namun menyambungkan tombol tersebut ke pin D8 (GPIO 15), lalu menekan tombol tersebut *tepat* pada detik di mana NodeMCU baru diberi daya?



Jika menggunakan rangkaian Active-High / *pull-down* dari Praktikum 2, tetapi tombol disambungkan ke D8 (GPIO 15), kemudian tombol ditekan tepat saat

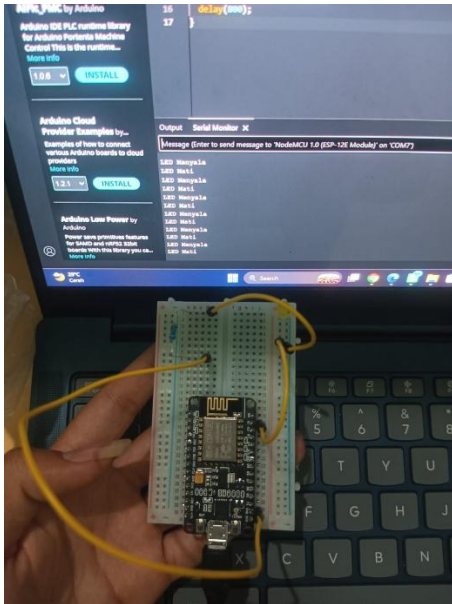
NodeMCU baru diberi daya, kondisi yang mungkin terjadi adalah NodeMCU gagal melakukan proses booting dengan normal. Hal ini karena GPIO15 termasuk pin yang digunakan dalam proses boot/strapping NodeMCU. Jika kondisi logika pada pin tersebut tidak sesuai ketika perangkat mulai menyala, proses boot dapat terganggu sehingga NodeMCU tidak menjalankan program seperti yang diharapkan.

Latihan 2 (Implementasi Kode): Modifikasi kode pada Praktikum 1 agar LED berkedip secara *tidak simetris*, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi delay() yang harus Anda ubah?



LED akan **menyala cepat selama 200 ms**, kemudian **mati selama 800 ms** sebelum menyala kembali.

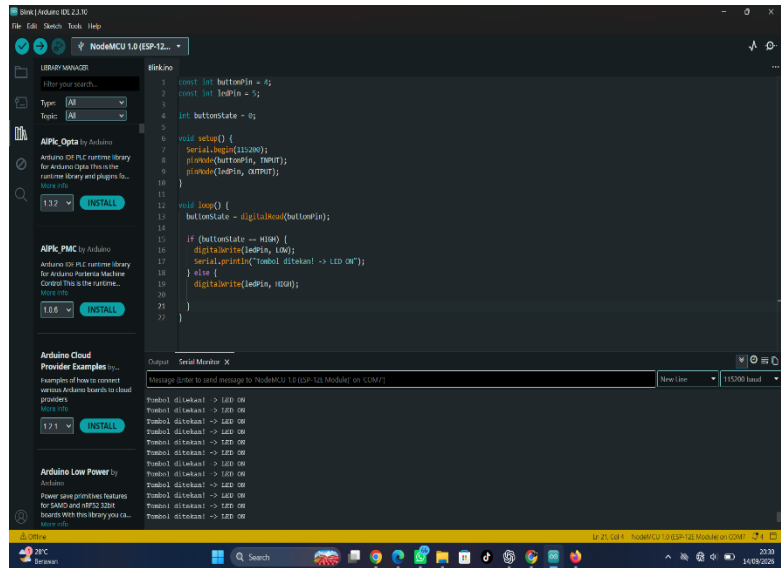
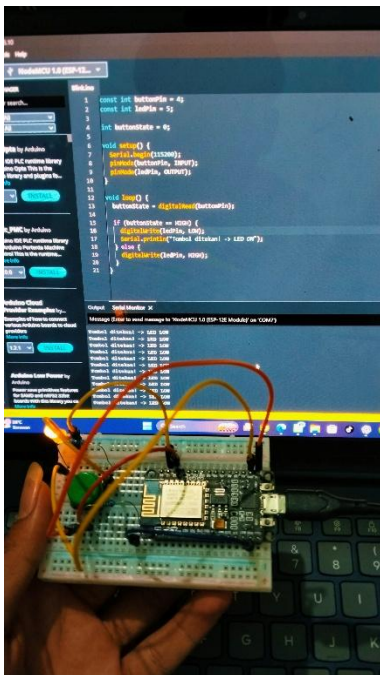
Latihan 3 (Analisis Sirkuit): Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami *error* kompilasi? Mengapa LED tidak menyala?



Jika kaki LED dipasang terbalik, yaitu **Anoda ke GND** dan **Katoda ke D1**, program **tidak mengalami error saat kompilasi**. Program tetap dapat dikompilasi karena posisi fisik kaki LED tidak berhubungan dengan sintaks program.

Namun, LED **tidak menyala** karena LED merupakan komponen yang memiliki arah aliran arus. Pada pemasangan tersebut, LED berada dalam kondisi **reverse bias**, sehingga arus tidak mengalir melalui LED dalam arah yang diperlukan untuk menghasilkan cahaya. Akibatnya, meskipun program memberikan perintah untuk menyalakan LED, LED tetap tidak menyala.

Latihan 4 (Pengembangan Algoritma): Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED **menyala**. Saat tombol DITEKAN, LED justru **mati**."



if (digitalRead(buttonPin) == LOW) { digitalWrite(ledPin, HIGH); } else { digitalWrite(ledPin, LOW); }

Jadi, ketika tombol **tidak ditekan**, pin membaca LOW sehingga LED **menyala**. Ketika tombol **ditekan**, pin membaca HIGH sehingga LED **mati**.

1.7 Tugas

Tugas ini harus dikerjakan secara mandiri untuk mengukur kemampuan integrasi dan logika pemrograman Anda.

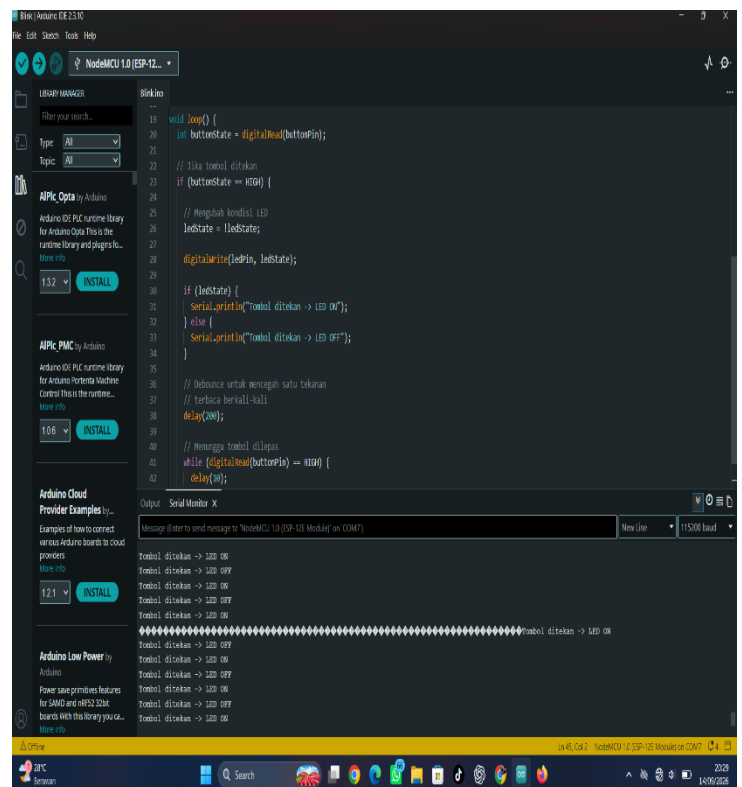
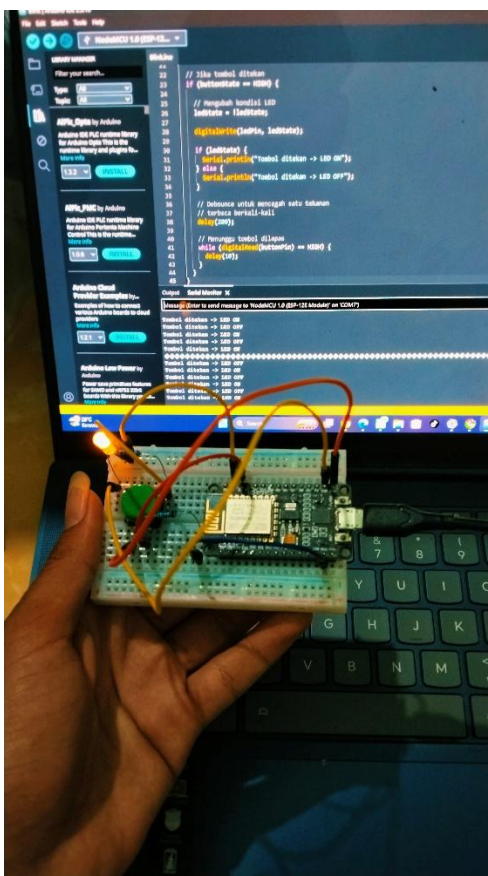
Modifikasi Sistem Sakelar *Toggle (Latching)*

- **Tujuan:** Mampu mengimplementasikan logika pelacakan status (*state tracking*) dan membedakan antara perilaku tombol *momentary* dengan *latching*.
- **Tingkat Kesulitan:** Menengah
- **Instruksi:**
 - i. Gunakan konfigurasi *hardware* yang persis sama dengan Praktikum 2 (Rangkaian *Pull-Down*).
 - ii. Modifikasi kode program sehingga tombol berfungsi layaknya sakelar lampu ruangan:
 - Saat sistem pertama dinyalakan, LED dalam keadaan mati.
 - Jika tombol ditekan 1 kali *lalu dilepas*, LED **menyala dan tetap menyala**.
 - Jika tombol ditekan 1 kali lagi *lalu dilepas*, LED **mati dan tetap mati**.
 - iii. *Petunjuk:* Gunakan variabel penyimpanan status (misalnya tipe boolean atau int) untuk melacak apakah lampu saat ini sedang ON atau OFF. Anda juga membutuhkan instruksi `delay()` singkat (sekitar 200 milidetik) tepat setelah mendeteksi tombol ditekan, untuk mengatasi fenomena pantulan mekanis (*debouncing* pada

tombol fisik).

- **Kriteria Keberhasilan:**

- Perubahan status (ON ke OFF, atau sebaliknya) hanya terjadi dengan *satu kali tekanan singkat* (bukan ditahan).
- Tidak ada fenomena *flickering* (lampu berkedip acak atau gagal mati) saat tombol ditekan.
- Program memiliki baris komentar yang menjelaskan bagaimana logika pelacakan *state* dan *debounce* bekerja.



Pertama, LED dipasang pada breadboard dengan **anoda LED dihubungkan ke GPIO 5 melalui resistor**, sedangkan katoda LED dihubungkan ke **GND**. GPIO 5 digunakan sebagai pin output untuk mengatur kondisi LED. Ketika NodeMCU memberikan logika HIGH, LED menyala, sedangkan ketika memberikan logika LOW, LED mati.

Selanjutnya, push button dipasang menggunakan rangkaian **pull-down**. Salah satu kaki tombol dihubungkan ke **3.3V**, sedangkan kaki lainnya dihubungkan ke **GPIO 4** dan resistor pull-down menuju **GND**. Dengan rangkaian tersebut, GPIO 4 membaca LOW ketika tombol tidak ditekan dan membaca HIGH ketika tombol ditekan.

Setelah seluruh perangkat terhubung, NodeMCU diberikan daya melalui kabel USB dan program Tugas 1.7 dijalankan. Pada kondisi awal, LED dibuat **OFF**. Ketika tombol ditekan dan dilepas satu kali, NodeMCU mengubah kondisi LED menjadi **ON** dan kondisi tersebut tetap tersimpan.

Ketika tombol ditekan dan dilepas untuk kedua kalinya, NodeMCU kembali

mengubah kondisi LED menjadi **OFF**. Proses tersebut terus berulang setiap kali tombol ditekan dan dilepas, sehingga rangkaian bekerja seperti **sakelar toggle (latching)**.

